Pause-Aware Depth Scheduling: Hiding Reasoning Latency Inside Conversational Silence for CPU-Only Edge Inference

B. Parswanadh*, Aryanandha V* *Department of Electronics and Computer Engineering, Amrita Vishwa Vidyapeetham, Bengaluru, India

***Abstract*—Self-speculative early-exit decoding allows a single large language model to draft tokens from its shallow layers and verify them with its deeper layers, avoiding the auxiliary-model and key-value (KV) cache incompatibility problems that afflict conventional two-model speculative decoding. Existing evaluations of this mechanism, however, are conducted almost exclusively on GPU-class, memory-compute-balanced hardware, and almost exclusively trigger deeper computation from token-level confidence *after* a prompt is complete. We identify two open, checkable questions this leaves unanswered: (i) whether self-speculative early exit retains its benefit on CPU-only, memory-bandwidth-bound consumer hardware with no discrete GPU, and (ii) whether the natural silence preceding the end of a human conversational turn (∼200–500 ms) can be exploited to begin deep-layer computation *before* the turn actually ends, hiding reasoning latency inside a resource – human pause time – that is otherwise unused. We term this combination Pause-Aware Depth Scheduling (PADS). We present the mechanism, a precise positioning against the closest prior systems – a two-model edge-cloud preemptive-drafting framework and a turn-detection cascade both also named after speculative decoding – and a pre-registered, falsifiable evaluation plan comprising seven feasibility ("Go/No-Go") experiments intended to be run before, not after, the full system is built. We report the outcome of the subset of these experiments and environment-verification steps completed to date, and describe the remaining evaluation protocol, hardware configuration, and threats to validity for the full study.**

***Index Terms*—edge inference, speculative decoding, early exit, on-device large language models, conversational AI, turn-taking prediction, CPU inference, low-power computing.**

I. Introduction

Conversational systems deployed on hardware with no discrete GPU – laptops, single-board computers, embedded assistants – face a structural trade-off: responses drawn only from shallow computation are fast but limited in reasoning depth, while responses that engage a model's full depth are capable but pay the full serial cost of every additional transformer layer. Self-speculative early-exit decoding, exemplified by LayerSkip [1], narrows this trade-off within a single model: early layers produce a draft token, and if verification against the full-depth output disagrees, the deeper layers are computed and the KV cache already built by the shallow pass is reused rather than discarded. Because both the shallow and deep passes belong to the same network, this approach sidesteps a problem that would otherwise be fatal to any design assuming two independently-sized models can exchange cached state: KV caches are tied to a model's own hidden dimension and layer count and are not interchangeable across differently-sized models.

Manuscript prepared as a working draft. Corresponding author: B. Parswanadh (venkataparswanadh@gmail.com).

Two properties of the existing early-exit literature motivate this work. First, essentially all reported speedups are measured on GPU-class accelerators, where inference is typically compute-bound; whether the same layer-skipping benefit survives on CPU-only, memory-bandwidth-bound hardware – the actual deployment target for most consumer edge devices – has, to our knowledge, not been systematically characterized. Second, the trigger for exiting early or continuing deeper is almost universally a per-token confidence signal evaluated *after* the model has already begun generating a response to a completed prompt. In a spoken or streaming-text conversational setting, however, a substantial and predictable interval – the pause before a human speaker finishes a turn, typically 200–500 ms [20], [21] – exists *before* generation would normally begin at all, and is presently left computationally idle by every system we are aware of.

This paper studies whether that interval can be used productively. We propose **Pause-Aware Depth Scheduling (PADS)**: a turn-taking predictor and a lightweight dialogue-act classifier jointly and conservatively decide, from a partial (in-progress) utterance, whether to begin extending computation to full model depth *during* the pause rather than only after the turn is confirmed complete. If the prediction is later confirmed, some or all of the expensive computation is already finished; if not, the speculative branch is discarded under the same correctness guarantee that standard speculative decoding already provides [2] – the final output distribution is unaffected regardless of whether the shallow or deep path is ultimately used.

A. Contributions and Scope

This paper makes the following contributions:

1) We identify and precisely bound a compound research gap at the intersection of self-speculative early exit and predictive conversational timing (Section II), including a direct differentiation against the two closest

prior systems we located, both of which independently adopted "speculative" naming for structurally different mechanisms.
2) We formalize a conservative, asymmetric trigger policy (Section III) under which false triggers are cheap by construction, and describe the two-tier scoping of the contribution – a safe, independently defensible characterization result, and an additive stretch mechanism – so that a negative result on the harder half of the claim remains a complete, reportable contribution rather than a failed project.
3) We specify a pre-registered, falsifiable evaluation protocol (Section IV) of seven feasibility experiments, each with an explicit kill condition, designed to invalidate the premise cheaply before implementation effort is committed to it.
4) We report verified environment and tooling results completed to date (Section V) and the outstanding evaluation plan.

We explicitly do not claim a working end-to-end system, final benchmark numbers, or a completed comparison against baselines at the time of this draft; Section V states precisely what has and has not been executed, consistent with the pre-registered evaluation design.

II. Related Work

A. Self-speculative and early-exit decoding

LayerSkip [1] trains a single model with layer dropout and a shared early-exit loss so that any layer can produce a usable output, then performs self-speculative decoding by drafting from a shallow exit and verifying against full depth with the same KV cache. Follow-on work refines *which* layers to skip and how the exit decision is made, including training-free dynamic-programming layer selection [3], input-adaptive layer sparsity [5], speculative-model-narrowed early-exit predictor search [4], and quantized-KV-cache self-speculation [6]. Critically, a 2026 systematic study [7] re-examines these assumptions on modern (post-2023) model families and reports that early-exit adaptability *decreases* across model generations as pretraining and alignment concentrate decision-relevant computation in later layers, and that dense transformers larger than approximately 20B parameters, together with models explicitly fine-tuned for early exit (as opposed to used off-the-shelf), retain the most exploitable redundancy. This finding directly informs our model-selection and Go/No-Go design (Section IV): it argues against assuming an arbitrary small instruction-tuned checkpoint will exhibit LayerSkip-like behavior without dedicated fine-tuning, and motivates measuring early-exit adaptability empirically before committing to a base model.

B. Routing and cascading between models

A separate, heavily populated line of work routes or cascades between an entire small model and an entire large model based on predicted query difficulty [8], [9], [10], a pattern our early framing of this project (as HNIA, prior to the present formulation) initially and mistakenly adopted. We do not build a router: PADS never selects between models, only between depths within one model, which avoids the KV-cache incompatibility that model-selection approaches do not need to solve and self-speculative approaches solve by construction.

C. The three closest prior systems

Three independently developed systems sit closer to PADS than the above and warrant explicit differentiation.

Venkatesha et al. [11] propose a speculative edge-cloud decoding framework in which a small draft model runs on an edge device (a Jetson Nano or Jetson Orin) and a large target model with auxiliary early-exit adapters runs on a cloud A100 server; early, partially-verified tokens from the server are used to *preemptively draft* the next batch of tokens on the edge device before the final verification round completes, exploiting otherwise-idle client time during the network round-trip. This is architecturally close to PADS in spirit – both convert idle time around a verification boundary into useful precomputation, and both preserve the correctness guarantee of standard speculative decoding – but differs on every dimension that matters for our claim: it requires two separately-trained models and a persistent network connection to a cloud server rather than a single self-speculative model running fully offline; the idle time it exploits is the client-server communication and verification latency of an already-complete prompt, not the human pause before a spoken or typed turn has finished; and its reported speedups (up to 35% over cloud autoregressive decoding, plus an additional 11% from preemptive drafting) are measured with an NVIDIA A100 in the loop, not on CPU-only hardware.

Ok et al. [12] propose SpeculativeETD, which – despite its name – applies the cheap-model-then-expensive-model cascade structure to the end-turn *detection* problem itself: a $\sim$200K-parameter on-device GRU continuously classifies speech as speaking or non-speaking, and only when it detects silence does a much larger server-side Wav2Vec2 model perform the harder classification of whether that silence is a mere pause or a genuine end of turn. This is the closest work by name and by structural analogy to speculative decoding, and we adopt its terminology for the pause/gap distinction. It does not, however, touch the language model's computation at all: its contribution stops at deciding *whether* to invoke the downstream LLM, and its heavier stage still depends on a server round-trip. PADS instead uses the same class of signal (a lightweight, continuously-running predictor over partial speech) to decide *how deep* into an already-invoked, fully local model's layers to compute, before the turn-detection decision is even finalized.

Srinivas et al. [13] propose a "talker"/"reasoner" architecture for conversational voice agents: a small, fast talker model immediately generates a contextually grounded response to mask the latency of a slower, more capable reasoner model running in parallel, then fluently incorporates the reasoner's streamed output as it arrives, closing the accuracy gap to within 6.3% of the frontier reasoner while maintaining millisecond-level response times. Like PADS, it targets dead time in a conversational agent's own response pipeline; unlike

PADS, it is architecturally a two-model system – exactly the router/cascade pattern we avoid by construction (Section II, Routing and cascading) – and the latency it hides is the agent's own post-prompt reasoning time while it is already speaking, not the human's pre-turn-end pause that PADS exploits before generation would otherwise begin at all.

To our knowledge, no prior system combines turn-level, pre-endpoint predictive triggering with model-*depth* extension within a single self-speculative model evaluated on CPU-only hardware. This is the compound gap PADS targets: each of the three systems above shares one property with PADS – exploiting otherwise-idle time, using a lightweight continuous predictive signal, or targeting a live conversational-latency problem – while diverging on model count, which idle window is exploited, or whether model depth is touched at all.

D. On-device inference characterization and turn-taking prediction

Independent surveys and measurement studies establish both halves of our premise as live, open questions rather than settled ones. On the hardware side, recent work directly measuring CPU-versus-GPU on-device inference reports thread-scaling plateaus consistent with a memory-bandwidth-bound regime under quantization [14], and memory-pressure-aware serving systems such as prima.cpp [15] demonstrate that consumer, resource-constrained hardware is an active target for large-model inference research, without characterizing self-speculative early exit specifically. On the turn-taking side, Voice Activity Projection [16] and its real-time, CPU-capable implementation [17], together with Google's jointly-optimized ASR/turn-taking predictor [18] (97% recall, 85% precision at 100 ms added latency) and Google's earlier predictive-ASR work that speculatively executes downstream processing before a final endpoint is confirmed [19], establish that lightweight, CPU-feasible, pre-endpoint predictive signals are a mature, validated capability independent of this work. PADS's contribution is not either capability in isolation, but their fusion with model-depth extension, characterized on hardware neither line of work targets.

III. System Design

A. Overview

The accompanying project repository includes a block diagram of the full pipeline, omitted here as a floating figure pending a camera-ready pass; the architecture is summarized in prose. Streaming partial speech or text feeds two lightweight, continuously-running components: a turn-taking predictor estimating $p_{\text{end}}(t)$, the probability the current turn will end within a short horizon, and a dialogue-act classifier estimating $p_{\text{deep}}(t)$, the probability that the eventual response will require full-depth reasoning rather than a shallow, low-latency response. A conservative trigger policy (Section III-C) combines both signals to decide whether to begin extending the self-speculative model to full depth during the pause, ahead of confirmed end-of-turn.

B. Depth extension mechanism

We adopt LayerSkip-style self-speculative decoding [1] as the underlying depth-extension mechanism rather than proposing a new one: a model trained with layer dropout and a shared early-exit loss produces a draft token from an early exit layer k, and, when triggered, continues computation through the remaining layers $k{+}1, \ldots, N$ to produce a verified token, reusing the KV cache already populated by the shallow pass. This choice is deliberate: it inherits a proven correctness guarantee (the final output distribution is identical whether or not early exit is used) and avoids re-deriving a mechanism that is already well-studied, concentrating the contribution on *when* the deep pass is triggered rather than *how* it is computed.

C. Trigger policy

Let τ_{end} and τ_{deep} be calibrated confidence thresholds. The pause-time trigger fires at time t iff

$$p_{\text{end}}(t) \geq \tau_{\text{end}} \ \wedge \ p_{\text{deep}}(t) \geq \tau_{\text{deep}}. \tag{1}$$

We use a conjunctive (AND) rather than disjunctive (OR) gate deliberately: a disjunctive gate roughly doubles the false-trigger rate for a comparable increase in true-trigger rate, and because a false trigger's cost is a discarded speculative branch rather than an incorrect output, the design goal is to keep false triggers *cheap and rare*, not to maximize recall on either signal individually. We treat the exact values of τ_{end} and τ_{deep} as parameters to be calibrated empirically (Section IV, Go/No-Go 3) rather than fixed a priori.

D. Expected latency model

Let T_{shallow} and T_{deep} denote the wall-clock cost of the shallow and full-depth passes respectively on the target hardware, and let T_{pause} denote the duration of the exploitable pause. Absent PADS, perceived latency after end-of-turn confirmation is T_{deep} whenever deep reasoning is required. Under PADS, if the trigger fires correctly at pause onset and $T_{\text{pause}} \geq T_{\text{deep}}$, the perceived post-confirmation latency approaches the residual $\max(0, T_{\text{deep}} - T_{\text{pause}})$; if $T_{\text{pause}} < T_{\text{deep}}$, the visible saving is bounded by T_{pause} itself. This immediately implies the premise is falsifiable on timing grounds alone – if T_{deep} on the target hardware exceeds typical pause durations by a wide margin, the achievable saving is small regardless of trigger accuracy, independent of any modeling choice. This is formalized as Go/No-Go 2 (Section IV).

E. Two-tier contribution structure

We separate the contribution into a *safe core* – a systematic characterization of self-speculative early-exit decoding on CPU-only, memory-bandwidth-bound hardware, with a dialogue-act-driven exit policy compared against the standard token-confidence exit policy used in prior work – and a *stretch layer*, the pause-time trigger of Section III-C itself. The safe core does not depend on the stretch layer succeeding and is independently reportable; the stretch layer is additive on top of the same infrastructure. This structure is a design decision, not an afterthought: it is intended to ensure that an honest negative

result on the harder, more novel half of the claim (e.g., if Go/No-Go 1 or 2 fails) still yields a complete, correctly-scoped contribution rather than requiring the project to be abandoned or the claim retroactively inflated.

IV. Evaluation Methodology

A. Target hardware and correctness requirement

All inference-time latency, throughput, memory, and energy measurements reported in the completed study will be collected on a Dell Latitude 5490 (8th-generation Intel CPU, Intel UHD 620 integrated graphics, 16 GB RAM, no discrete GPU) running models in GGUF format via llama.cpp [22]. Training and fine-tuning (continued pretraining of the early-exit backbone, dialogue-act classifier training) will use a local RTX 4070 and rented cloud GPU capacity, but no number reported as an inference/latency/energy result will originate from GPU hardware, to keep the central hardware claim honest and independently reproducible on commodity hardware. Every configuration must satisfy the same correctness property as standard speculative decoding: a discarded speculative branch must not alter the final output token distribution relative to non-speculative full-depth decoding.

B. Pre-registered feasibility experiments (Go/No-Go tests)

Table I lists seven falsification experiments, each with an explicit kill condition and the action taken if that condition is met, executed prior to full system implementation. This ordering is deliberate: several of the seven target assumptions that, if false, would invalidate downstream engineering effort at low cost if caught early and high cost if caught late.

C. Baselines

All baselines will be reproduced locally on the target hardware rather than cited from other papers' reported numbers, to avoid conflating hardware effects with methodological ones: plain (non-speculative) decoding; standard two-model speculative decoding [2]; plain cascading/routing between a small and large model; and LayerSkip's own published configuration [1] reproduced on this hardware.

D. Metrics

Time-to-first-token, tokens/sec, peak RAM, energy draw (RAPL/turbostat or USB power-meter fallback per Go/No-Go 6), acceptance rate of the self-speculative mechanism, and false-trigger / true-trigger rate of the pause-time policy. All reported values will be means over multiple runs with 95% confidence intervals, collected under a fixed CPU governor with no competing background load, following a standardized warm-up-and-discard protocol informed by Go/No-Go 4.

V. Status and Verified Results

Consistent with the pre-registered design in Section IV, this draft reports only what has been executed and verified to date; it does not report placeholder, simulated, or projected benchmark numbers as if they were measured results. All numbers in this section were collected on a development machine (Alienware m16 R2, Intel Core Ultra 7 155H, 22 threads, 15 GB RAM) while the target Dell Latitude 5490 was not yet available; they characterize the pipeline and its behavior, not the final target-hardware result, and are re-run on the target machine before being treated as reportable in any final version of this paper. The checkpoint used throughout is the smallest officially released LayerSkip model, `facebook/layerskip-llama3.2-1B` [1] (1B parameters, quantized to Q4_K_M for the llama.cpp-based measurements), chosen to fit comfortably within this machine's RAM budget with headroom for the eventual classifier and turn-taking components.

- **Toolchain.** llama.cpp was built for a CPU-only target (`-DGGML_NATIVE=ON`, no CUDA/Metal backend, npm-based web UI disabled). Compilation completed with zero errors (∼81 s, 21 threads). The compiled `llama-cli` binary was confirmed to correctly load a real GGUF model and generate text, not merely to compile.
- **Software components implemented and unit-tested.** The trigger-policy module (Equation 1) passed a six-test unit suite covering threshold boundaries, out-of-range inputs, and outcome classification. A benchmark-harness module implementing the aggregation and structured-logging requirements of Section IV was validated via self-test with synthetic timing data (excluded from any reported result).
- **Go/No-Go 1 (bandwidth-vs-compute): FAILED, as pre-registered.** Tokens/sec measured across 1–8 threads (`llama-cli`, Q4_K_M): 16.7, 30.2, 36.5, 43.5, 47.2, 48.7, 49.3, 49.1. Throughput scales near-linearly to 4 threads, then flatlines from 5 threads onward – the kill condition of Table I, row 1, firing cleanly. Per the pre-committed mitigation for this outcome, this is reframed as a legitimate characterization result rather than a project failure: notably, this already used the smallest released LayerSkip checkpoint (1B parameters), smaller than the 3B fallback the mitigation plan proposed testing, and the bandwidth-bound regime still holds – a robust, not marginal, confirmation of the memory-bandwidth-bound premise this paper's central bet depends on.
- **Go/No-Go 2 (pause-duration feasibility): PARTIAL.** The decode-step-timing half was measured directly: wall-clock time for one depth-extension step (drafted tokens → full-depth verification) across $n = 147$ real steps was mean 285.1 ms, median 263.2 ms (interquartile range 256.6–274.4 ms, long tail to 1062.3 ms), using the LayerSkip reference (PyTorch/`transformers`) implementation rather than the faster quantized llama.cpp path – likely a pessimistic estimate of real deployment-time cost. A preliminary comparison against previously-published pause-duration ranges (not a specific-corpus measurement; real Switchboard/CallHome pause statistics remain pending corpus access) suggests the timing premise is plausible but pause-length-dependent: only 4.8% of measured steps complete within 200 ms, but 91.2% complete within 300 ms. The real-corpus half of this experiment is

TABLE I
PRE-REGISTERED GO/NO-GO FEASIBILITY EXPERIMENTS

#	Experiment	Method	Kill condition / action if triggered
1	Bandwidth-vs-compute regime	Vary thread count 1–8 on a quantized 7–9B GGUF model, measure tokens/sec	Throughput plateau past 4–5 threads indicates a memory-bandwidth-bound regime; reframe as a characterization result and/or reduce model size
2	Pause-duration feasibility	Compare real pause-duration statistics (Switchboard/CallHome) against measured per-step depth-extension latency on target hardware	Median pause shorter than one meaningful depth-extension step invalidates the timing premise as designed; redesign granularity rather than re-tune thresholds
3	Off-the-shelf acceptance rate	Run a public LayerSkip checkpoint unmodified on conversational-style prompts (`generation_strategy=self_speculative`), measure acceptance rate	Too low even before quantization damage; flag before custom training begins. (A logit-similarity-based early-exit adaptability score [7] across candidate checkpoints is a planned refinement, not yet implemented; see Section V.)
4	Thermal stability	30-minute sustained inference loop, log CPU frequency/temperature	Significant throttling; adopt fixed warm-up period and discard-first-runs as a permanent benchmarking protocol
5	RAM budget	Load intended models/components at intended quantization, measure peak RSS with OS overhead	Pipeline does not fit in 16 GB with headroom; reduce model size before adding engineering complexity
6	Energy tooling availability	Attempt RAPL via `turbostat`/`powertop` on target hardware	Tooling reports no usable data; adopt an external USB in-line power meter as the energy-measurement method
7	Fresh literature scoop check	Recurring biweekly targeted search for the specific fusion described in Section III	Exact fusion published; reposition framing around whichever half of the claim remains differentiated

not yet complete.

- **Go/No-Go 3 (off-the-shelf acceptance rate): PASSED.** The unmodified `facebook/layerskip-llama3.2-1B` checkpoint, run with its own self-speculative generation strategy (`exit_layer=3`, `num_speculations=6`, greedy decoding) on a small conversational-style prompt set, achieved a mean acceptance rate of 86.4% across 8 samples (range 75.1–100%). This is well above the kill threshold and broadly consistent with GPU-reported figures for this mechanism, despite CPU-only, quantized, small-model conditions.
- **Go/No-Go 4 (thermal stability): not yet executed.** This experiment requires an uninterrupted 30-minute sustained-load run; scheduling it is pending.
- **Go/No-Go 5 (RAM budget): PARTIAL.** The measurement script is verified correct, reporting real system-level RAM figures (15.13 GB total, 7.34 GB available at time of measurement). The full pipeline-level peak-RSS measurement (summed across inference, classifier, and turn-taking processes) is pending those components being run concurrently.
- **Go/No-Go 6 (energy tooling): PASSED.** All three candidate methods produced plausible, reproducible, non-zero readings on this hardware: `turbostat` (package power 13.88–14.36 W, package temperature 68–81°C across three samples), `powertop`, and a direct RAPL sysfs read.
- **Go/No-Go 7 (literature scoop): PASSED.** A search across the six pre-registered queries found no prior system combining turn-level pre-endpoint triggering with model-depth extension within a single self-speculative model on CPU-only hardware, corroborating Section II.

VI. THREATS TO VALIDITY

Construct validity. The early-exit adaptability finding of [7] indicates that not every candidate base model will exhibit the redundancy self-speculative decoding depends on. Go/No-Go 3, as implemented and executed (Section V), measures off-the-shelf acceptance rate directly rather than this adaptability score; a finer-grained, logit-similarity-based per-model adaptability measurement remains a planned refinement, not yet implemented, for distinguishing checkpoint suitability *before* committing to an acceptance-rate run on any given candidate.

Internal validity. Thermal throttling and background system load on consumer laptop hardware can confound latency measurements if not controlled; Go/No-Go 4 and the fixed-protocol requirement in Section IV are intended to address this directly rather than discover it post hoc. Go/No-Go 4 has not yet been executed (Section V), so this mitigation is a design commitment at present, not yet a verified one.

External validity. All results reported in Section V were collected on a development machine (Alienware m16 R2), not the target Dell Latitude 5490, whose availability is still pending – this is itself an open external-validity gap, disclosed explicitly rather than papered over, and no number in this draft should be read as target-hardware-representative until re-collected there. Once collected on the target device, results from one specific laptop model may still not generalize to all CPU-only consumer hardware; we regard this as a characterization of a representative device class rather than a universal claim, consistent with prior single-device edge-inference studies [11].

Conclusion validity / novelty risk. Section II identifies the two closest prior systems and the precise dimensions of difference; Go/No-Go 7 treats the possibility of being independently scooped on the compound claim as an ongoing, monitored risk rather than a one-time check.

VII. CONCLUSION AND FUTURE WORK

We have described Pause-Aware Depth Scheduling, a mechanism for triggering self-speculative early-exit depth extension during the pause preceding a conversational turn's end, positioned precisely against the two closest prior systems we

identified and scoped into an independently defensible safe-core characterization plus an additive stretch contribution. A pre-registered set of seven falsification experiments is designed to invalidate the premise cheaply before deep implementation investment, consistent with treating this as a checkable empirical claim rather than a vision statement. Future work comprises executing the outstanding Go/No-Go experiments on the target hardware, implementing and benchmarking the safe-core pipeline against reproduced baselines, and – contingent on an internal go/continue checkpoint – implementing and evaluating the pause-time stretch mechanism itself.

ACKNOWLEDGMENT

[TBD: Guide and evaluator acknowledgments, Open Lab I/II course context.]

REFERENCES

[1] M. Elhoushi *et al.*, "LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.
[2] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023.
[3] L. Chen *et al.*, "CLaSp: In-Context Layer Skip for Self-Speculative Decoding," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025.
[4] J. Xu *et al.*, "SpecEE: Accelerating Large Language Model Inference with Speculative Early Exiting," in *Proc. 52nd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2025.
[5] H. Xia, Y. Li, J. Zhang, C. Du, and W. Li, "SWIFT: On-the-Fly Self-Speculative Decoding for LLM Inference Acceleration," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
[6] "QuantSpec: Self-Speculative Decoding with Hierarchical Quantized KV Cache," arXiv:2502.10424, 2025.
[7] R. Wei, R. Du, H. Yu, D. Tiwari, J. Li, Z. Xu, and H. Wang, "The Diminishing Returns of Early-Exit Decoding in Modern LLMs," arXiv:2603.23701, 2026.
[8] I. Ong *et al.*, "RouteLLM: Learning to Route LLMs with Preference Data," arXiv:2406.18665, 2024.
[9] "MixLLM: Dynamic Routing in Mixed Large Language Models," arXiv:2502.18482, 2025.
[10] "Lookahead Routing for Large Language Models," arXiv:2510.19506, 2025.
[11] Y. Venkatesha, S. Kundu, and P. Panda, "Fast and Cost-effective Speculative Edge-Cloud Decoding with Early Exits," arXiv:2505.21594, 2025.
[12] H. Ok, S. Yoo, and J. Lee, "Speculative End-Turn Detector for Efficient Speech Chatbot Assistant," arXiv:2503.23439, 2025.
[13] V. Srinivas, Z. Englhardt, V. Iyer, and S. Patel, "Thinking While Speaking: Inference-Time Knowledge Transfer for Responsive and Intelligent Conversational Voice Agents," arXiv:2511.07397, 2025.
[14] "Challenging GPU Dominance: When CPUs Outperform for On-Device LLM Inference," arXiv:2505.06461, 2025.
[15] "Prima.cpp: Fast 30–70B LLM Inference on Heterogeneous and Low-Resource Home Clusters," arXiv:2504.08791, 2025.
[16] E. Ekstedt and G. Skantze, "Voice Activity Projection: Self-Supervised Learning of Turn-Taking Events," arXiv:2205.09812, 2022.
[17] K. Inoue *et al.*, "Real-time and Continuous Turn-taking Prediction Using Voice Activity Projection," arXiv:2401.04868, 2024.
[18] S. Chang, B. Li, T. N. Sainath, C. Zhang, T. Strohman, Q. Liang, and Y. He, "Turn-Taking Prediction for Natural Conversational Speech," arXiv:2208.13321, 2022.
[19] "Personalized Predictive ASR for Latency Reduction in Voice Assistants," arXiv:2305.13794, 2023.
[20] T. Stivers *et al.*, "Universals and Cultural Variation in Turn-Taking in Conversation," *Proc. Natl. Acad. Sci.*, vol. 106, no. 26, pp. 10587–10592, 2009.
[21] M. Heldner and J. Edlund, "Pauses, Gaps and Overlaps in Conversations," *J. Phonetics*, vol. 38, no. 4, pp. 555–568, 2010.
[22] G. Gerganov *et al.*, "llama.cpp," https://github.com/ggml-org/llama.cpp, accessed 2026.